Absolutely. Based on your actual work, especially **FastAPI + React + microservices + PostgreSQL/MongoDB + RAG/LLM + YouTube analytics + company intelligence**, you should answer these questions with **project-specific examples**, not generic textbook answers.

Here's an interview-ready structure.

---

1. What did you build?

Strong answer

> In my current role, I work as a Full Stack Developer, mainly with **Python/FastAPI on the backend and React on the frontend**.
>
> One of the major systems I worked on was an **AI-powered company intelligence platform**. The platform collects information from different sources such as company documents, news, YouTube videos, and other external data sources, and uses **RAG and LLM-based analysis** to generate insights about companies.
>
> On the backend, I designed and developed multiple FastAPI microservices for areas such as:
>
> - Company management
> - Document management
> - RAG-based document chat
> - Company perception analysis
> - YouTube analytics
> - Mutual Fund intelligence
> - Subscription/payment and coupon management
>
> I used **PostgreSQL/MySQL for transactional data and MongoDB where flexible document-based querying was useful**.
>
> For the AI side, I worked with **LLMs, LangChain, vector databases, embeddings and RAG pipelines**.
>
> On the frontend, I developed React-based dashboards that consume these APIs and present the intelligence and analytics to users.

If they ask for one concrete example

Use your **Company Intelligence / RAG platform** as the main project.

---

2. Why did you choose FastAPI?

Don't just say "FastAPI is fast."

Good answer:

> We chose FastAPI because the application was heavily API-driven and required a scalable backend architecture.
>
> FastAPI gave us:
>
> 1. **Async support** for I/O-heavy operations such as external API calls and LLM services.
> 2. **Pydantic validation**, which helped us maintain strict request and response contracts.
> 3. **Automatic OpenAPI/Swagger documentation**, which made frontend and service integration easier.
> 4. Good performance because it is built on **Starlette and Pydantic**.
> 5. A clean dependency injection mechanism, which helped with database sessions, authentication and reusable services.
>
> Another important reason was that our AI/RAG ecosystem was already Python-based, so FastAPI allowed us to integrate libraries such as LangChain, embedding models and LLM SDKs naturally.

Follow-up: "Why not Django?"

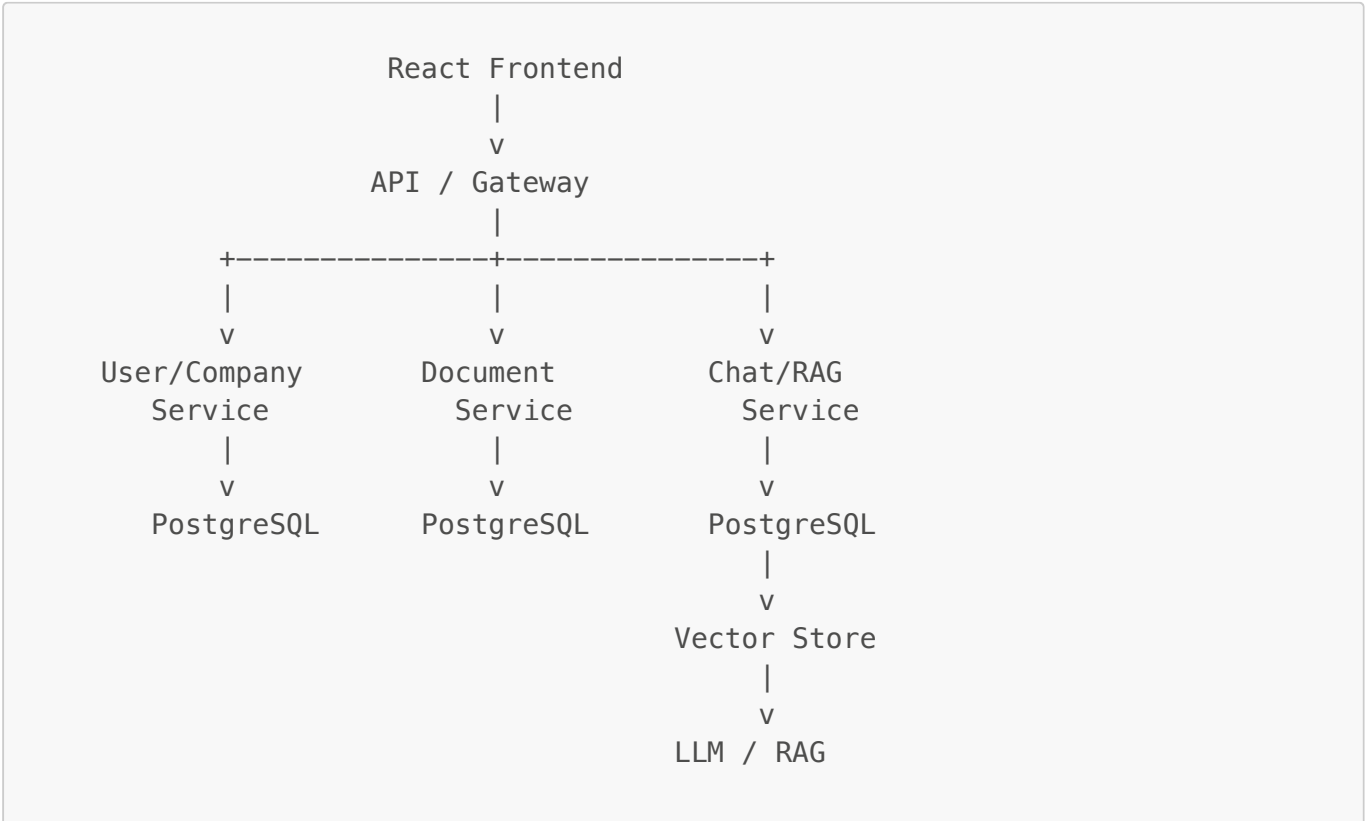
> Django would also have been a valid choice, especially for a monolithic application with a large admin layer. But our backend was primarily API and microservice oriented, so FastAPI gave us a lighter architecture and better integration with our asynchronous AI and external-service workflows.

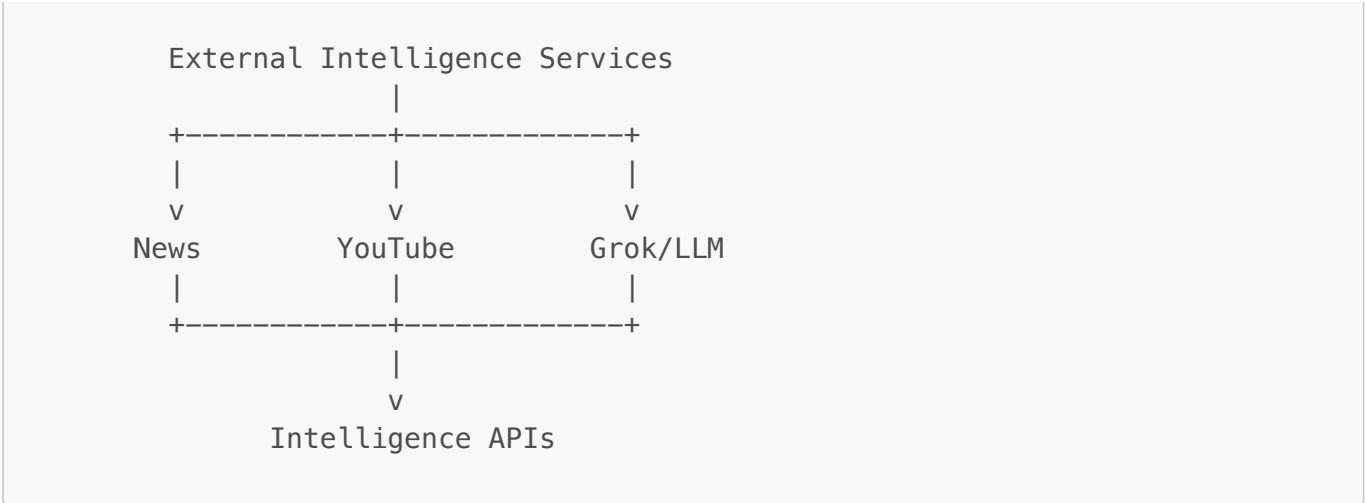
---

3. Explain your architecture

This is one of the most important questions.

You can explain your architecture like this:





Explain it verbally

> We follow a service-oriented/microservice architecture.
>
> The React application communicates with backend APIs. Each major business capability is separated into a service, such as company, document, chat, perception and analytics.
>
> Each service follows a layered structure:
>
> **Router → Service → Query/Repository → Model**
>
> The router handles HTTP concerns, the service contains business logic, and the query/repository layer handles database operations.
>
> For AI workloads, we have an additional flow:
>
> **API → Service → Connector → External provider → Processing → Response**
>
> For example, the YouTube service communicates with the YouTube API through a dedicated connector/client rather than putting external API logic directly inside the FastAPI route.

That last point is **very good in a senior interview**.

---

4. Why separate Client, Connector and Service?

This is something you can explain from your actual architecture.

> I intentionally separated the external integration into different layers.
>
> For example:
>
> ```
> Router
>    ↓
> Service
>    ↓
> Connector
>    ↓
> ```

```
Client
   ↓
YouTube API
```

The **client** handles low-level API communication.

The **connector** handles integration-specific logic such as authentication and adapting the external API response.

The **service** handles our actual business logic.

This separation makes the code easier to test and also makes it easier to replace an external provider later.

Then give your YouTube example:

> For YouTube, I had a `youtube_client`, `youtube_connector` and `youtube_perception_service`. This prevented Google API-specific implementation details from leaking into the business layer.

---

5. What challenges did you face?

Don't give only one.

Prepare **5–6 real challenges**.

Challenge 1 — RAG context/token limitations

> One major challenge was controlling the amount of context sent to the LLM.
>
> Initially, retrieving too many documents or large chunks caused context/token-limit issues.
>
> I solved this by improving the retrieval pipeline:
>
> - Better chunking
> - Metadata filtering
> - Limiting top-K results
> - Removing duplicate content
> - Selecting only relevant chunks
> - Controlling the final prompt size
>
> This reduced unnecessary context and improved both performance and response quality.

Challenge 2 — LangChain document/metadata issues

You have actually encountered this.

> During the RAG implementation, we encountered issues where the retrieved LangChain documents didn't contain the metadata structure we expected. For example, we had code expecting a `score`

> field while the returned document object didn't expose it directly.
>
> Instead of depending on assumptions about the framework object, I normalized the retrieval response into our own internal structure before passing it to the business layer.
>
> That made our application less coupled to LangChain's internal response format.

That's a **strong senior-level answer**.

Challenge 3 — Multiple databases

> We had multiple services and different data requirements, so we didn't force everything into a single database.
>
> We used relational databases for transactional and structured data, while MongoDB was useful for flexible company-related data and filtering.
>
> The important design decision was to keep database ownership at the service level instead of allowing every service to directly access every database.

If they ask:

"How do microservices communicate?"

Say:

> For synchronous operations, services communicate through REST APIs. For asynchronous workloads, I would use an event/message-based approach where appropriate. The important principle is that one service should not directly manipulate another service's database.

---

6. Challenge: External API failures

Use your YouTube/Grok work.

> We depend on external providers such as YouTube and LLM APIs. These introduce problems like rate limits, timeouts, malformed responses and temporary failures.
>
> I handled this by isolating external API calls behind connectors, validating responses, adding appropriate timeout/error handling and keeping provider-specific logic outside the business layer.
>
> This also made it easier to change providers without rewriting the entire application.

---

7. Challenge: YouTube analytics architecture

This is a very good project-specific example.

You can say:

> For YouTube analytics, one challenge was that a company can have multiple videos, and users should not manually configure those video IDs from the dashboard.
>
> So I separated the configuration responsibility from the user-facing analytics.
>
> An admin maintains the mapping:
>
> ```
> Company
>    |
>    +-- Video ID 1
>    +-- Video ID 2
>    +-- Video ID 3
> ```
>
> The company dashboard simply requests analytics for the configured videos.
>
> This gives us better control, avoids user errors and makes the analytics pipeline easier to automate.

---

8. What performance improvements did you make?

This is where you should demonstrate seniority.

Database

> I optimized database access by avoiding unnecessary queries, selecting only required fields, adding appropriate indexes and avoiding N+1 query patterns.

API

> For API performance, I used asynchronous operations where they provided value, especially for I/O-heavy external integrations.

External APIs

> I reduced unnecessary calls by reusing data where possible and separating data collection from user-facing requests.

RAG

> In the RAG pipeline, the biggest performance improvement came from controlling retrieval size and context size. Instead of sending large amounts of retrieved content to the LLM, I filtered and ranked the relevant chunks before constructing the final prompt.

Frontend

> On the React side, I avoid unnecessary renders using techniques such as component decomposition, memoization where justified, controlled API fetching and appropriate state management.

9. "How did you measure performance?"

This is a **very important follow-up**.

Don't say:

> "It became faster."

Instead:

> I look at metrics such as API response time, database query execution time, external API latency, number of database queries per request, payload size and error rate.
>
> For AI workflows, I also look at retrieval latency, number of retrieved chunks, prompt/context size and LLM response latency.

If you have actual production numbers, use them. **Don't invent numbers.**

10. What trade-offs did you make?

This question separates mid-level from senior candidates.

Microservices

> Microservices give us independent deployment and scalability, but they also introduce network communication, deployment complexity, observability and distributed failure scenarios.
>
> We accepted that complexity because the system had multiple independent business capabilities and integrations.

PostgreSQL + MongoDB

> Using multiple databases gives us flexibility, but it increases operational complexity.
>
> We chose this because the data access patterns were different enough to justify it.

Async

> Async improves throughput for I/O-bound operations, but it doesn't automatically make CPU-heavy workloads faster. So I use it mainly for network and I/O operations rather than treating it as a universal performance solution.

RAG vs sending everything to LLM

> Sending all documents to the LLM would be simpler but expensive, slow and limited by context size.

> RAG adds retrieval complexity, but it significantly reduces the amount of context we need to send and allows us to ground responses in our own data.

---

11. Why PostgreSQL?

> PostgreSQL was a good fit for structured transactional data such as users, companies, chats, messages, subscriptions and orders.
>
> We needed relationships, constraints, transactions and reliable consistency, which relational databases handle very well.

If they ask **"Why not MongoDB?"**

> MongoDB is excellent when the data is document-oriented and schema flexibility is important. But for strongly relational entities such as orders, subscriptions and payments, PostgreSQL provides better relational integrity and transactional guarantees.

---

12. Why MongoDB?

Use your company filtering example.

> We used MongoDB in areas where the data structure was more flexible and where document-style querying and filtering were useful.
>
> For example, company-related data can have varying attributes and filtering requirements. MongoDB allowed us to model that naturally without creating a large number of relational tables for every variation.

---

13. How did you design your FastAPI code?

You can show this:

```
app/
├── routers/
│   └── company.py
│
├── services/
│   └── company_service.py
│
├── queries/
│   └── company_query.py
│
├── models/
│   └── company_model.py
│
```

```
├── schemas/
│   └── company_schema.py
│
├── clients/
│   └── youtube_client.py
│
├── connectors/
│   └── youtube_connector.py
│
└── core/
    └── config.py
```

Then:

> I try to keep the router thin. It should primarily validate input, call the service and return the response.
>
> Business logic belongs in the service.
>
> Database operations belong in the query/repository layer.
>
> External integrations belong in clients/connectors.

That's a very good answer for your experience.

---

14. Authentication/security

If they ask how you secure the APIs:

> I generally separate authentication from authorization.
>
> Authentication verifies who the user is, typically using JWT/token-based authentication.
>
> Authorization determines what that user can access based on roles and permissions.
>
> I also validate request payloads using Pydantic, avoid trusting client-provided identifiers for authorization decisions, keep secrets in environment/secret management systems, and apply appropriate database and API-level access controls.

---

15. "Tell me about one difficult bug"

Use your RAG bug.

Answer:

> One difficult issue I faced was in the RAG pipeline where the retrieval result structure was different from what our application expected.

> Some code was expecting a relevance `score` directly on the LangChain document object, but the actual object didn't expose it that way.
>
> I first traced the complete flow from vector retrieval to the application layer rather than fixing the error blindly.
>
> I found that we were coupling our business logic directly to the framework's object structure.
>
> I fixed it by normalizing the retrieval output into our own internal representation containing the content, metadata and relevance information we actually needed.
>
> This solved the immediate issue and also made the retrieval layer more maintainable.

---

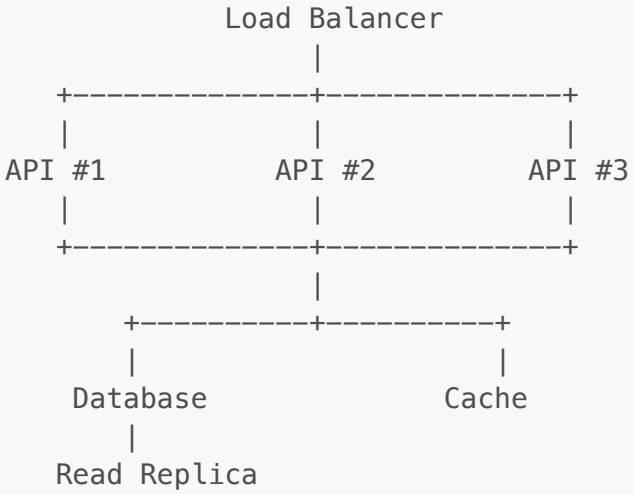
16. "How would you scale your system?"

This is very likely for your profile.

Say:

> I would scale it horizontally by running multiple instances of each stateless FastAPI service behind a load balancer.
>
> For database-heavy workloads, I would focus on indexing, connection pooling, query optimization and potentially read replicas.
>
> For expensive AI and external API operations, I would move long-running tasks to asynchronous workers and queues rather than making the user request wait.
>
> I would also introduce caching for frequently requested data and use monitoring to identify bottlenecks.
>
> For RAG specifically, I would optimize embedding generation, vector retrieval, chunking and LLM calls independently.

Architecture:



```
        Heavy/Async Work
              |
            Queue
              |
         Worker Pool
              |
        LLM / External APIs
```

---

17. "What would you improve if you rebuilt it?"

Excellent senior answer:

> I would improve three areas.
>
> **First, asynchronous processing.**
>
> Some expensive operations such as document processing, embedding generation and external intelligence collection should be handled through background workers rather than synchronous API requests.
>
> **Second, observability.**
>
> I would introduce centralized logging, distributed tracing and metrics across services so that a slow request can be traced across service boundaries.
>
> **Third, caching and data freshness.**
>
> I would explicitly define which data needs real-time freshness and which data can be cached. This would reduce unnecessary external API and LLM calls.

---

18. Your 2-minute project answer

If the interviewer says:

"Tell me about your current project."

Use this:

> Currently I'm working on an AI-powered company intelligence platform. I'm primarily responsible for backend development using Python and FastAPI, along with React-based frontend development.
>
> The platform collects information from different sources such as company documents, news and YouTube, and uses RAG and LLM-based processing to generate company insights.
>
> Architecturally, we follow a service-oriented approach where different business capabilities are separated into services. Within each service, I follow a layered architecture with routers, services, queries/repositories, models and schemas.

> PostgreSQL is used for structured transactional data, while MongoDB is used where flexible document-based querying makes more sense. For AI workloads, we use vector retrieval, embeddings and LLMs.
>
> One of the challenges I worked on was controlling RAG context size. Retrieving too much information caused token and latency issues, so I improved chunking, filtering, deduplication and top-K retrieval before sending information to the LLM.
>
> I also worked on external integrations such as YouTube and LLM providers. I separated clients and connectors from business logic so that external-provider failures and changes wouldn't affect the rest of the application.
>
> From a performance perspective, I've worked on reducing unnecessary database queries, optimizing retrieval, controlling payload/context size and using asynchronous I/O where appropriate.
>
> The main trade-off we've made is accepting some microservice and distributed-system complexity in exchange for independent scalability, deployment and separation of business responsibilities.

Your strongest project examples to prepare

For your interviews, I'd prioritize these **six stories**:

Area	Story to prepare
🧠 AI/RAG	Context/token problem + retrieval optimization
🏗️ Architecture	FastAPI microservices + layered architecture
🗄️ Database	PostgreSQL + MongoDB decision
🔌 Integration	YouTube/Grok connector architecture
📊 Product	YouTube company analytics
⚡ Performance	DB optimization + RAG optimization + async I/O

And for **every story**, remember this structure:

Problem → Why it mattered → Your decision → Implementation → Challenge → Solution → Result → Trade-off

That structure will let you handle most **"What did you build / why / challenge / performance / trade-off"** questions naturally instead of sounding like you're reciting prepared answers.